# SIBER SIAGA 2025

# Capture the Flag: Code Combat!

# Finale




**Write-up by 0LDBEES**

Amin | Melvin | Ariff

# Table of Contents

The Starting Point ..... 3
Interceptosis ..... 4
Key to the Other Side ..... 6
U S B ..... 9
What's the Password ..... 10
The Dark Blog ..... 11

## The Starting Point

### 📚 Challenge Description

One of our agents managed to locate one of the suspects. We lost contact before he could tell us where he was. This snapshot was the only documentation he could send. Can you find where this place is?

Note: Use the google map name if you have found the location.

Flag format: SIBER25{main_street-town_name-state_name-country}

### ✨ Walkthrough




At first we examine the picture and we can see that it located in russia as the store name in russian language. By using chatgpt, I manage to get the place correctly.

https://chatgpt.com/share/68db8566-3db8-800c-be9b-a11f558f6b7c

### 🏳 Flag

`SIBER25{ulitsa_sovetskaya-chaykovsky-perm_krai-russia}`

# Interceptosis

## 📚 Challenge Description

Our team was able to intercept the suspect's message to his group. They may have sent something sensitive in the process, discover a way to decrypt the communication and figure out what their chat is about.

## ✨ Walkthrough

From the pcap file given, we can export the http objects. The important files among them are the cat picture that shows the encryption logic, the group chat log file, and the encrypted firmware.

The cat pic gives the hint for decrypting the group chat logs. The chat is encrypted using a fixed key and nonce. After some googling, i found a blog that might be useful for us from https://book-of-gehn.github.io/articles/2018/12/04/Fixed-Nonce-CTR-Attack.html.

I gave the link to ChatGPT asking it to perform the frequency attack and some bruteforcing on the XOR key. After a few iterations, the group chat file is readable, and it exposed the key for decrypting the encrypted firmware. It revealed the IP address, the port number, and the endpoint to get the key files.

Once the key file is downloaded, I uploaded both the file and encrypted firmware to ChatGPT to perform the decryption. The file returned was a zip file containing the flag and also the code for the next challenge.

| user | plaintext_recovered |
|---|---|
| solarizzer | init solarizzer |
| WaderPav | init WaderPaw |
| PenguinDo | init PenguinDog |
| solarizzer | H am starting a full diagnostic on all the compromised network nodes. Early reports show that our entuy poinbs are holning etwonf wutuoxt ans dxteftioq from pheir outlasdd ivneusian lysdkqs |
| WaderPav | Dxcellent. The initial phase is complete. The transportation grid is now showing major failures acrost all mwjor city kreas8 |
| WaderPav | Uheir emergency response teams are completely overwhelmed, running in circles chasing the ghost signaks we pzanted. Tho pubzif tsanoii iata leeys dre oure gifberish. At r beyocifub cwaoc |
| solarizzer | Food. Chaos is the objective. Let them feel the ground shifting beneath their feet. We are demonstratnng thab we contrel ths serx antxrdes ol tueiw ciky. Thiw is not ibhtt mwtry; gt vs qlsuz phyez zzd zzecw/datihtithkgi |
| solarizzer | Laintain active surveillance on their network recovery teams. I want a full report on their every movb, no mwtter how ymall8 |
| PenguinDo | Ve have a problem. A local one. I'm at the East-side depot to prep the hardware for phase two, but I fm compzetely locaed oct+ |
| PenguinDo | H pushed the new firmware update to the door's RFID scanner last night, but it is not accepting any oa our acthenticatcon cwras. |
| solarizzer | Dxplain. This is an unacceptable setback. That hardware is critical to our timeline. What did you do so the pirmware? |
| PenguinDo | Gor security, I encrypted the firmware with AES-CTR before uploading it. It is our standard operationfl procsdure to pxevenb wevdrsy-xnjineexinz ic thz unit ms physicillxx cahnbred |
| PenguinD | Uhe problem is, I was juggling three different tasks, and I honestly cannot recall the exact key I usbd. I hwve tried eur sbakdasd noiayion ef veyv, bjt nothmng is wozknog. Lrr fibe vs uhzemtiqklq ngelyfd z w sdzs dfz |
| WaderPav | QenguinDog, you idiot. You actually forgot the key? Unbelievable. |
| WaderPav | Oevermind that. Just send a GET request to our cloud server. |
| WaderPav | Ht is at 5.223.49.127. |
| solarizzer | Tse the method our leader demonstrated for extracting the key from the key file you got. |
| WaderPav | Rurely you have not forgotten that basic exploitation technique yet, have you? |
| PenguinDo | Nh, right. That method. The request should be sent on port 4510 if I remember correctly. |
| WaderPav | Xes, that is the correct port. And remember the API is at /getkey. |
| WaderPav | @lso, do not forget that the key is always Base64 encoded. |
| WaderPav | Ht is also quite funny how the firmware is encrypted so insecurely. |
| solarizzer | Xeah with the nonce being just the first 12 bytes of the key. |
| WaderPav | H swear, if the higher ups knew we were still using such a predictable nonce generation scheme, we world soox find ouryelvee ln were yehp treubqe. |
| PenguinDo | Uhat's it! Man, thank you. I knew it was something simple. Okay, I can retrieve the key easily. I'll oave ths scanner eperabijnam ir iwhnty ginhtev. Sprry fov the alazm) |
| solarizzer | Uhis is your only warning, PenguinDog. Document your procedures. Such a trivial mistake could have jehpardizsd the entcre ofewathon2 Zey it nonx, dnd set's gat back of tbhedmvr. Tferz wybp le ia eoiq engxrmo |

Group Chat Recovery: https://chatgpt.com/share/e/68db8697-0770-8002-9838-22d48beabe48

Firmware Decryption:https://chatgpt.com/share/e/68db8661-96e4-8002-aac1-a0a1c12fbcf9

### 🏳 Flag

`SIBER25{WuZZl3_l0v3S_AES}`

# Key to the Other Side

## 📚 Challenge Description

The suspect's door is protected by an RFID security device. To minimize the impact of our breach, we must find a way to acquire access. Perhaps the suspect might have a way to communicate with his members on how to gain entry to his base?

Note: If you have found the key, make sure is all lowercase

## ✨ Walkthrough

Our team was able to intercept the suspect's message to his group. They may have sent something sensitive in the process, discover a way to decrypt the communication and figure out what their chat is about.

```
rydzze /mnt/c/U/Hp/Downloads/SiberSiaga25_FinalsTI ❯ file SesameDoor_dist                                    9s
SesameDoor_dist: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x
86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=81544629ae0a32249a48b0bc5134fb7b1455adea, stripped

rydzze /mnt/c/Users/Hp/Downloads/SiberSiaga25_FinalsTI ❯ strings SesameDoor_dist | grep python
pyi-python-flag
Failed to set python home path: %s
Failed to pre-initialize embedded python interpreter!
Failed to set python home path!
Failed to allocate PyConfig structure! Unsupported python version?
Failed to start embedded python interpreter: %s
Failed to start embedded python interpreter!
```

That said, we decrypt the encrypted file and receive a zipped file with a file inside, SesameDoor_dist. After that, we run the file command to confirm it's an ELF executable. We scan strings inside the ELF and see Python-related markers so it's a PyInstaller-packed Python app.

```
rydzze /mnt/c/U/Hp/Downloads/SiberSiaga25_FinalsTI ❯ python pyinstxtractor.py SesameDoor_dist                 10s
/mnt/c/Users/Hp/Downloads/SiberSiaga25_FinalsTI/pyinstxtractor.py:23: SyntaxWarning: invalid escape sequence '\p'
  C:\path\to\exe\>python pyinstxtractor.py <filename>
[+] Processing SesameDoor_dist
[+] Pyinstaller version: 2.1+
[+] Python version: 3.11
[+] Length of package: 17179751 bytes
[+] Found 55 files in CArchive
[+] Beginning extraction...please standby
[+] Possible entry point: pyiboot01_bootstrap.pyc
[+] Possible entry point: pyi_rth_inspect.pyc
[+] Possible entry point: SesameDoor_dist.pyc
[!] Warning: This script is running in a different Python version than the one used to build the executable.
[!] Please run this script in Python 3.11 to prevent extraction errors during unmarshalling
[!] Skipping pyz extraction
[+] Successfully extracted pyinstaller archive: SesameDoor_dist

You can now use a python decompiler on the pyc files within the extracted directory
```

We run a PyInstaller extractor (pyinstxtractor) to unpack the embedded archive and dump .pyc files. Next, we decompile the .pyc files using pylingual.io to get the original Python source and open the main module.

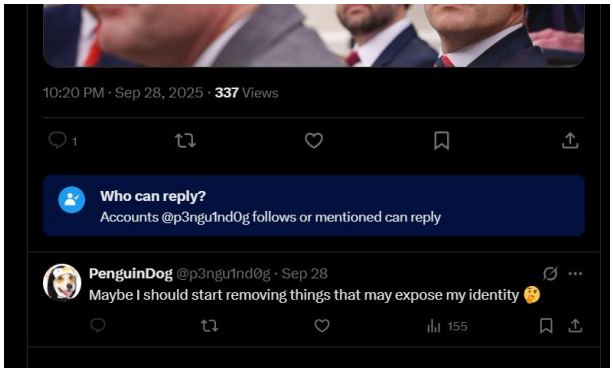
In short, the script reads the tag's block 0 and block 4. It builds a 16-byte key from an **env secret, My_Favourite_Animal**, computes a transformation XOR16(block0) and checks whether that equals block 4. If they match, authentication succeeds and the flag is revealed. The XOR16 function repeats the ASCII key to 16 bytes which repeat the 8-byte key twice, then XORs each byte of block 0 with the corresponding key byte, producing a 16-byte result ... moving on to the OSINT part

From Challenge 1 we extracted three names from the conversation. We searched for each name on X to see whether they had an account. Eventually we discovered that **PenguinDog** had an X account. While reviewing that account we found a post in which the user said they had started deleting some of their posts.

```
solarizzer : d9e3ffeb55838b317dca16b915e7eba6

WaderPaw : d9e3ffeb5583af3f75ce16800eea

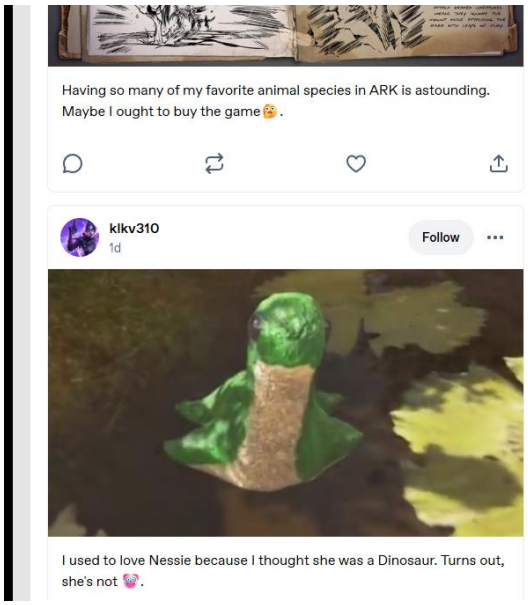
PenguinDog : d9e3ffeb5583a83b7fcc11b901d9e1b3
```




Using archive.today, we recovered the deleted post — it contained a username for another platform.



With that username, we ran Sherlock to search for matching accounts across social networks and found a Tumblr account.




Investigating the Tumblr page, we discovered the key **"dinosaur"**. We used a script to extract the value needed to write to the RFID card and retrieved the flag.

We take ASCII of **dinosaur**, repeat it to get **dinosaurdinosaur** (16 bytes). We XOR each byte of the given block-0 with that repeated key to compute block-4.

For block-0: 9A 50 C0 5F 55 6C F3 A2 9A 6A 64 75 60 78 1D B7

With key **dinosaurdinosaur**, block-4 becomes:

FE 39 AE 30 26 0D 86 D0 FE 03 0A 1A 13 19 68 C5

To write it with a Proxmark we authenticate to the card's sector, write the 16-byte payload into block 4, then read back to verify. Last but not least, scan the card for the flag :)))

### 🏳 Flag

**SIBER25{H4CK3D!}**

# USB

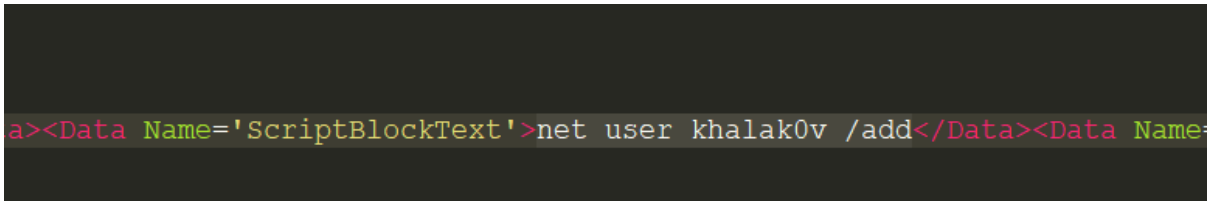
### 📚 Challenge Description

The suspect left his USB. Can you figure out the name of this suspect?

Flag Format: SIBER25{username}

### ✨ Walkthrough

We extract the USB disk image and found some event log files from it. The eventlog file was then converted into XML format. And after some exploration, we found a command “net user khalak0v /add” located in the Operational.XML file which showed the username.




### 🏳 Flag

`SIBER25{khalak0v}`

## What's the Password

### 📚 Challenge Description

The suspect may have left several sheets at this base, I wonder if one of them contains the password for entry to The Dark Blog.

Flag Format: SIBER25{password}

### ✨ Walkthrough

The potential password was in one of the sticky notes in the base. We identified the potential password and list it down. We identified the correct password by try error each password on the DarkBlog website with the username obtained from previous challenge.

### 🏳️ Flag

**FLAG**

`SIBER25{Kh@l4k0v!2025#A7}`

# The Dark Blog

### 📚 Challenge Description

It appears that the group has a private forum where they discuss their activity and APT operations. We will need the credentials to access the forum. The forum may reveal important information on the group's planned attack on the client. Help us find out the tools that they may use for this cyberwarfare.

Flag Format: SIBER25{MODEL_NAME-MODEL_NUMBER}

### ✨ Walkthrough

There are a lot of blog posted on the Dark Blog website. After going through the blogs, the “Eyes on the skies” catch our attention as it explicitly mentions about stealing the famous TODAK. On top of that, there is a similar picture of the blog post at the base from the previous challenge. The challenge provider gave us a hint that the answer are not restricted to the ctf playground, hence we took a look at the booths available. And there is indeed one Todak booth a the exact plane model with a tag that contains the model number, which matches the flag format.




### 🏳 Flag

`SIBER25{TODAK-EX13Z37F}`